

Deployment Guide

Follow these steps to run SkillBridge on a server or production host.

Automated Nginx and SSL setup

After pointing your domain DNS records to the server, run the installation wizard to configure Nginx and request Let's Encrypt certificates:

```
./install.sh
```

The script prompts for the environment and, in production mode, the domain name. To run non-interactively you can provide arguments:

```
./install.sh production yourdomain.com
```

In either case the script updates the domain placeholders in `nginx/conf.d` and uses `certbot` (or `acme.sh`) to generate certificates at the paths referenced in `nginx/conf.d/ssl.conf`.

Configure environment variables

1. **Backend** – copy `backend/.env.example` to `backend/.env` and set:

- `PORT` – typically `5000` unless changed.
- `APP_DOMAIN` – your production domain (e.g. `yourdomain.com`).
- `SUPPORT_EMAIL` – address used for outbound messages.
- `FRONTEND_URL` – set this to the full URL of your frontend. You can specify multiple domains separated by commas. For example:

Example using a custom domain and server IP

`APP_DOMAIN=yourdomain.com FRONTEND_URL=https://{APP_DOMAIN},http://147.93.121.45`

Do not prefix with "FRONTEND_URL=" when using docker-compose environment variables.

```
```
```

If the frontend and backend are on different subdomains, also set `COOKIE_DOMAIN` so the authentication cookie can be shared. Example:

```
```bash
```

```
COOKIE_DOMAIN=${APP_DOMAIN}
```

When running over plain HTTP but using different subdomains (e.g. staging

environments), also add:

```
```bash
COOKIE_SECURE=false
COOKIE_SAMESITE=None
```

To enable password recovery via email, provide SMTP settings or configure them later through the `/api/email-config` endpoint or the admin dashboard at /dashboard/admin/settings/email-config`. At a minimum the backend requires the following variables:`

```
```bash
SMTP_HOST=smtp.mailtrap.io
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=your_smtp_username
SMTP_PASS=your_smtp_password
```
```

To disable sending emails (OTP codes will be logged instead), set:

```
```bash
DISABLE_EMAILS=true
```
```

The default templates include your logo and a footer. OTP codes expire

after **\*\*15 minutes\*\***. You can customize the app name, logo and contact email from `/dashboard/admin/settings/app` so outbound emails show your branding.`

This value is used for CORS and socket.io connections. If it still points to `http://localhost:3001` you may see `Network Error` or CORS errors when logging in from the deployed site.`

2. **Frontend** – for Docker Compose production builds, set variables in `frontend/.env.production` :

```
Point the frontend to your backend including the /api prefix
NEXT_PUBLIC_API_BASE_URL=https://{APP_DOMAIN}/api
```

The root `.env` is intended for local development and defaults `NEXT_PUBLIC_API_BASE_URL` to `http://backend:5002/api` for internal container communication. Remove or override this file in production so the frontend uses your public HTTPS domain.

Without this variable the frontend defaults to `/api` which may point to the wrong server when deployed.

## Production example: eduskillbridge.net

To deploy the official site, copy `.env.example` to `.env` and `backend/.env.production.example` to `backend/.env.production` . Fill in the database credentials, JWT and refresh token secrets, and SMTP settings with values managed outside of version control. Configure the public URLs:

```
APP_DOMAIN=eduskillbridge.net
FRONTEND_URL=https://eduskillbridge.net
```

```
NEXT_PUBLIC_API_BASE_URL=https://eduskillbridge.net/api
NEXT_PUBLIC_SOCKET_URL=https://eduskillbridge.net
```

Keep these files out of git and store the secrets in your hosting environment or secrets manager.

After updating these files, rebuild the Docker images or restart the server so that the environment changes take effect.

## Run database migrations

Before starting the backend server or after pulling updates, apply any pending database migrations:

```
npm --prefix backend run migrate
```

Running migrations separately keeps `startServer()` lightweight and ensures the database schema matches the application's expectations.

## Validate your deployment

**Before inviting users:** Run these production smoke tests to make sure the environment is stable.

- Check the API health endpoint with `curl https://<domain>/api/health` and confirm it returns `{"status":"ok"}`.
- Sign in to the frontend, navigate to `/dashboard/admin`, and confirm pages load without console errors.
- Send a password reset email or test notification to verify SMTP credentials are correct.
- Run `docker compose ps` (or your process manager equivalent) to confirm backend, frontend, and worker processes are running.
- Review `docker compose logs backend` and `docker compose logs nginx` for warnings about migrations, CORS, or TLS.

## Preserve uploaded media

The admin panel lets you upload a logo and favicon under `/dashboard/admin/settings/app`. These files are stored inside the backend container under `/app/uploads`. To keep them after a container restart, mount the `backend/uploads` folder from the host. In `docker-compose.yml` add:

```
backend:
 volumes:
 - ./backend/uploads:/app/uploads
```

This ensures custom branding files persist across deployments.

## Preserve database and pgAdmin data

PostgreSQL stores its data in the `postgres_data` volume and pgAdmin uses `pgadmin_data`. In production, avoid running `docker compose down -v` as the `-v` flag removes these volumes and wipes the database and pgAdmin settings. Use `docker compose stop` or `docker compose down` without `-v` to retain your data. Manually deleting the `postgres_data` or `pgadmin_data` volumes will also erase data. Consider mounting these volumes to external storage or setting up regular backups to protect critical information.

## Next.js image domains

Uploads served from the backend domain are now automatically whitelisted for Next.js Image. The hostname and port are derived from `NEXT_PUBLIC_API_BASE_URL` at build time. If you need to allow additional domains you can still extend `remotePatterns` in `frontend/next.config.mjs`.

## Troubleshooting

### Login page requests `http://localhost:5000`

If you deploy the frontend and see network errors pointing to `http://localhost:5000/api` it means the build did not have `NEXT_PUBLIC_API_BASE_URL` set. Update `frontend/.env.local` with the correct backend URL and rebuild/restart the frontend container so the new value is picked up.

### CORS errors when logging in

If the browser console shows messages like `No 'Access-Control-Allow-Origin'` or `ERR_NETWORK` during login, your backend is rejecting the frontend's origin. Edit `backend/.env` and ensure the `FRONTEND_URL` variable lists the exact domain of your deployed site without a trailing slash. Include both the `www` and non-`www` variants if you use them. For example:

```
FRONTEND_URL=https://${APP_DOMAIN},https://www.${APP_DOMAIN}
```

Restart the backend so the updated CORS settings take effect. If CORS errors occur when requesting a password reset, ensure the `FRONTEND_URL` contains your frontend's domain. The API only sends CORS headers for domains listed there. If you use Nginx or another reverse proxy, ensure it does **not** add its own `Access-Control-Allow-Origin` header. Duplicates will cause browsers to reject the response even if both values are identical.

### Home page shows "Failed to load tutorials" or "Failed to load categories"

These messages mean the frontend cannot reach the API. Verify that `NEXT_PUBLIC_API_BASE_URL` in `frontend/.env.local` points to your backend URL including the `/api` prefix. Also confirm the backend server is running and that the `FRONTEND_URL` in `backend/.env` includes your frontend domain.

### Login redirects repeatedly

If you briefly see the home page then get bounced back to the login form with browser console messages about CORS, your backend is not allowing the frontend's origin. Double check that `FRONTEND_URL` in `backend/.env` matches the deployed domain exactly and restart the backend. The `NEXT_PUBLIC_API_BASE_URL` in `frontend/.env.local` must also point to the backend including the `/api` prefix. When the frontend and API are on different subdomains, set `COOKIE_DOMAIN` in `backend/.env` to the shared base domain (e.g. `.${APP_DOMAIN}`) so the authentication cookie is available to both sites. If the site uses HTTP, also set:

```
COOKIE_SECURE=false
COOKIE_SAMESITE=None
```

This ensures the refresh token cookie is sent across subdomains without HTTPS.